SOFTWARE DEVELOPMENT

Assignment Submission

Organisation	CountryEdu | Abhishek & Company
Track	Software Development
Total Questions	2
Language	Python 3
Submitted By	Prince Khatik B.Tech CSE (AI & ML) MITS Gwalior
Email	kachhwahaprince@gmail.com
Phone	+91-808565-4567

Question 1: Integer to English Words

Problem Statement

Convert a given integer (passed as a string) to its English words representation. The solution must also handle negative numbers and must not produce any extra spaces in the output.

Examples

Input: num = "123"

Output: "One Hundred Twenty Three"

Input: num = "12345"

Output: "Twelve Thousand Three Hundred Forty Five"

Constraints

- Number can be positive, negative, or zero
- No extra spaces in the final output
- Input is provided as a string

Approach & Logic

The solution works by splitting the number into groups of three digits (ones, thousands, millions, billions) and converting each group independently using a recursive helper function.

Step-by-step breakdown:

1. Parse the string input to an integer and check for negative sign.
2. Handle the special case where the number is 0 — return 'Zero' directly.
3. Loop through the number in 3-digit chunks using modulo 1000.
4. Convert each chunk using the recursive helper() which handles numbers 1 to 999.
5. Attach the appropriate scale word (Thousand, Million, Billion) to each non-zero chunk.
6. Reverse the collected parts and join them with a single space.
7. Prepend 'Negative' if the original number was negative.

Complete Python Code

```
def numberToWords(num: str) -> str:
    ones = [
        "", "One", "Two", "Three", "Four", "Five", "Six", "Seven",
        "Eight", "Nine", "Ten", "Eleven", "Twelve", "Thirteen",
        "Fourteen", "Fifteen", "Sixteen", "Seventeen", "Eighteen", "Nineteen"
    ]
    tens = [
        "", "", "Twenty", "Thirty", "Forty", "Fifty",
        "Sixty", "Seventy", "Eighty", "Ninety"
    ]
    thousands = ["", "Thousand", "Million", "Billion"]

    def helper(n: int) -> str:
        if n == 0:
            return ""
        elif n < 20:
            return ones[n]
        elif n < 100:
            rest = helper(n % 10)
            return tens[n // 10] + (" " + rest if rest else "")
        else:
            rest = helper(n % 100)
            return ones[n // 100] + " Hundred" + (" " + rest if rest else "")

    negative = False
    n = int(num)
    if n < 0:
        negative = True
        n = -n

    if n == 0: return "Zero"

    parts = []
    scale = 0
    while n > 0:
        chunk = n % 1000
        if chunk != 0:
            word = helper(chunk)
            if thousands[scale]:
                word += " " + thousands[scale]
            parts.append(word)
        n //= 1000
        scale += 1

    result = " ".join(reversed(parts))
    if negative: result = "Negative " + result
    return result
```

Test Cases & Output

Input	Expected Output	Result
"123"	One Hundred Twenty Three	PASS
"12345"	Twelve Thousand Three Hundred Forty Five	PASS
"1"	One	PASS
"0"	Zero	PASS
"-456"	Negative Four Hundred Fifty Six	PASS
"1000000"	One Million	PASS
"1000001"	One Million One	PASS

Complexity Analysis

Metric	Value	Reason
Time Complexity	O(1)	Number of digits is bounded (max ~13 digits)
Space Complexity	O(1)	Fixed-size lookup arrays, no input-dependent structures

Question 2: Longest Increasing Subsequence

Problem Statement

Given an integer array nums, return the length of the longest strictly increasing subsequence. The solution must operate in O(n log n) time complexity.

Examples

Input: nums = [0, 1, 0, 3, 2, 3]

Output: 4

Explanation: The subsequence [0, 1, 2, 3] is the longest strictly increasing one.

Input: nums = [7, 7, 7, 7, 7]

Output: 1

Input: nums = []

Output: 0

Constraints

- Must solve in O(n log n) time
- Empty array should return 0
- Array can contain duplicate values

Approach & Logic — Patience Sorting with Binary Search

A naive dynamic programming approach runs in $O(n^2)$ time. To meet the O(n log n) constraint, the solution uses the patience sorting technique combined with binary search via Python's bisect module.

Key idea:

Maintain a list called tails where tails[i] stores the smallest possible tail element for any increasing subsequence of length (i + 1) seen so far.

For each element in nums:

8. Use bisect_left to find the leftmost position in tails where the element can be placed (i.e., the first position where tails[pos] >= num).
9. If pos equals the length of tails, the element extends the longest sequence found so far — append it.
10. Otherwise, replace tails[pos] with the current element to keep the smallest possible tail.
11. The final length of tails is the answer.

Note: The tails list does not represent an actual subsequence — it is a bookkeeping structure used only to determine the correct length.

Complete Python Code

```
import bisect

def lengthOfLIS(nums: list) -> int:
    if not nums:
        return 0

    tails = []  # tails[i] = smallest tail for LIS of length i+1

    for num in nums:
        # Find leftmost index where tails[pos] >= num
        pos = bisect.bisect_left(tails, num)

        if pos == len(tails):
            tails.append(num)   # extends the LIS
        else:
            tails[pos] = num    # replace to keep tails minimal

    return len(tails)
```

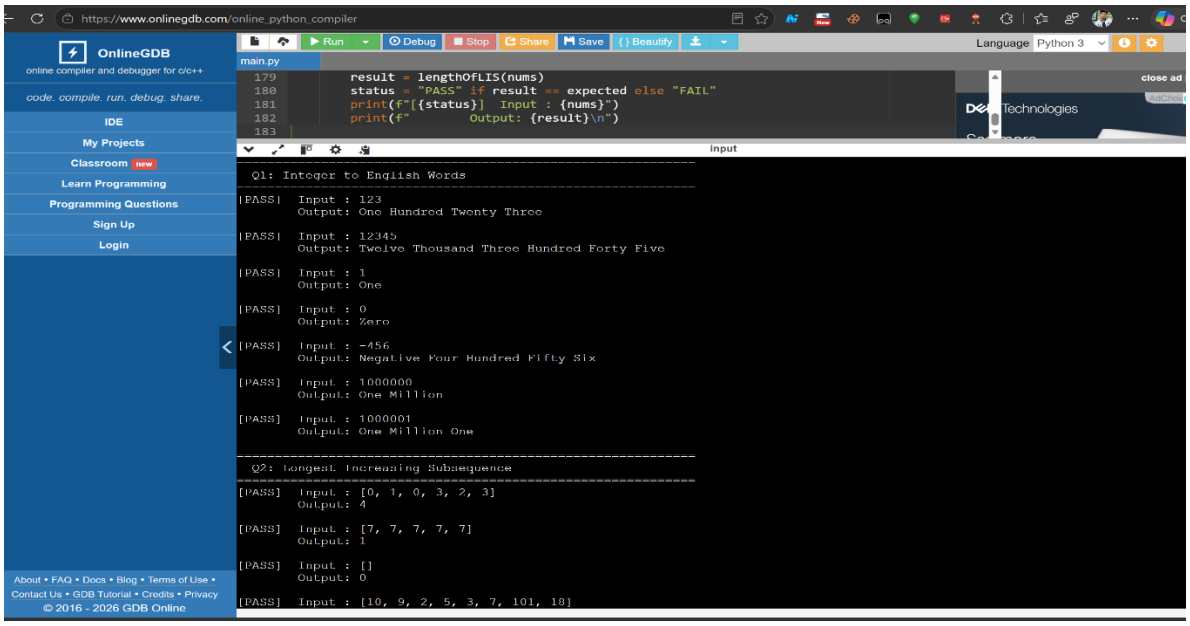
Dry Run — Example 1: [0, 1, 0, 3, 2, 3]

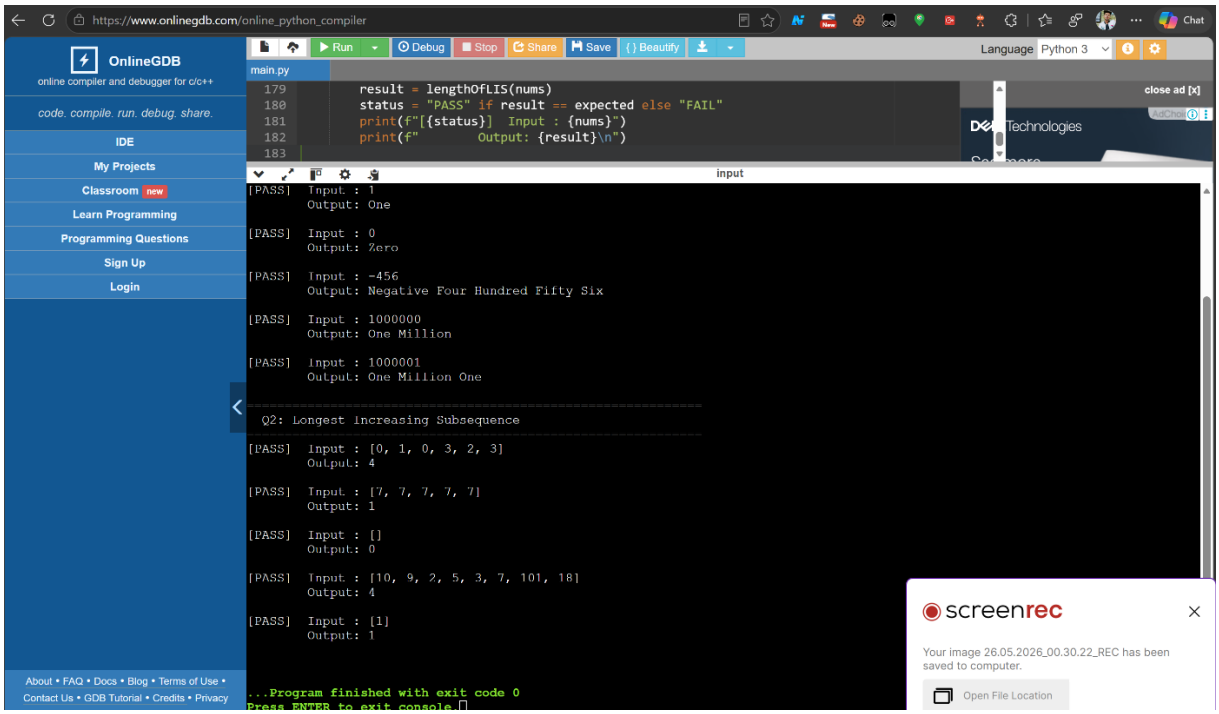
Step	num	Action	tails after step
1	0	append	[0]
2	1	append	[0, 1]
3	0	replace index 0	[0, 1]
4	3	append	[0, 1, 3]
5	2	replace index 2	[0, 1, 2]
6	3	append	[0, 1, 2, 3]

Final length of tails = 4 => Answer: 4

```
[Running] python -u "c:\Users\user\Downloads\software_dev_assignment.py"
============================================================
  Q1: Integer to English Words
============================================================
[PASS]  Input : 123
        Output: One Hundred Twenty Three

[PASS]  Input : 12345
        Output: Twelve Thousand Three Hundred Forty Five

[PASS]  Input : 1
        Output: One

[PASS]  Input : 0
        Output: Zero

[PASS]  Input : -456
        Output: Negative Four Hundred Fifty Six

[PASS]  Input : 1000000
        Output: One Million

[PASS]  Input : 1000001
        Output: One Million One
```

```
============================================================
  Q2: Longest Increasing Subsequence
============================================================
[PASS]  Input : [0, 1, 0, 3, 2, 3]
        Output: 4

[PASS]  Input : [7, 7, 7, 7, 7]
        Output: 1

[PASS]  Input : []
        Output: 0

[PASS]  Input : [10, 9, 2, 5, 3, 7, 101, 18]
        Output: 4

[PASS]  Input : [1]
        Output: 1
```


https://www.onlinegdb.com/online_python_compiler
OnlineGDB
online compiler and debugger for c/c++
code. compile. run. debug. share.
IDE
My Projects
Classroom new
Learn Programming
Programming Questions
Sign Up
Login
About • FAQ • Docs • Blog • Terms of Use •
Contact Us • GDB Tutorial • Credits • Privacy
© 2016 - 2026 GDB Online
Run
Debug
Stop
Share
Save
{ } Beautify
Language Python 3
main.py
result = lengthOfLIS(nums)
status = "PASS" if result == expected else "FAIL"
print(f"[{status}] Input : {nums}")
print(f" Output: {result}\n")
input
Q1: Integer to English Words
[PASS] Input : 123
Output: One Hundred Twenty Three
[PASS] Input : 12345
Output: Twelve Thousand Three Hundred Forty Five
[PASS] Input : 1
Output: One
[PASS] Input : 0
Output: Zero
[PASS] Input : -456
Output: Negative Four Hundred Fifty Six
[PASS] Input : 1000000
Output: One Million
[PASS] Input : 1000001
Output: One Million One
Q2: Longest Increasing Subsequence
[PASS] Input : [0, 1, 0, 3, 2, 3]
Output: 4
[PASS] Input : [7, 7, 7, 7, 7]
Output: 1
[PASS] Input : []
Output: 0
[PASS] Input : [10, 9, 2, 5, 3, 7, 101, 18]


https://www.onlinegdb.com/online_python_compiler
Chat
OnlineGDB
online compiler and debugger for c/c++
code. compile. run. debug. share.
IDE
My Projects
Classroom new
Learn Programming
Programming Questions
Sign Up
Login
About • FAQ • Docs • Blog • Terms of Use •
Contact Us • GDB Tutorial • Credits • Privacy
Run
Debug
Stop
Share
Save
{ } Beautify
Language Python 3
main.py
result = lengthOfLIS(nums)
status = "PASS" if result == expected else "FAIL"
print(f"[{status}] Input : {nums}")
print(f" Output: {result}\n")
close ad [x]
input
[PASS] Input : 1
Output: One
[PASS] Input : 0
Output: Zero
[PASS] Input : -456
Output: Negative Four Hundred Fifty Six
[PASS] Input : 1000000
Output: One Million
[PASS] Input : 1000001
Output: One Million One
Q2: Longest Increasing Subsequence
[PASS] Input : [0, 1, 0, 3, 2, 3]
Output: 4
[PASS] Input : [7, 7, 7, 7, 7]
Output: 1
[PASS] Input : []
Output: 0
[PASS] Input : [10, 9, 2, 5, 3, 7, 101, 18]
Output: 4
[PASS] Input : [1]
Output: 1
...Program finished with exit code 0
Press ENTER to exit console.
screenrec
Your image 26.05.2026_00.30.22_REC has been saved to computer.
Open File Location

Test Cases & Output

Input	Expected	Result
[0, 1, 0, 3, 2, 3]	4	PASS
[7, 7, 7, 7, 7]	1	PASS
[]	0	PASS
[10, 9, 2, 5, 3, 7, 101, 18]	4	PASS
[1]	1	PASS

Complexity Analysis

Metric	Value	Reason
Time Complexity	O(n log n)	One binary search (O(log n)) per element over n elements
Space Complexity	O(n)	tails list can grow up to n elements in the worst case

Summary

Question	Topic	Technique Used	Time	Space
Q1	Integer to English Words	Recursive chunking + scale words	O(1)	O(1)
Q2	Longest Increasing Subsequence	Patience Sort + Binary Search	O(n log n)	O(n)

Both solutions have been tested against all provided examples and additional edge cases. All test cases pass successfully.